

OPMS Security Documentation

Authentication, authorization, secrets, edge TLS, and known risks

Generated: 2026-09-22

Diagrams rendered with Mermaid 11 · Secrets redacted

Contents

08-security/01-security-overview.md
02-architecture/07-authentication-authorization.md
04-api/02-authentication.md
03-functional/authentication.md
06-developer-deployment/19-security.md
06-developer-deployment/05-environment-variables.md

OPMS Security Documentation

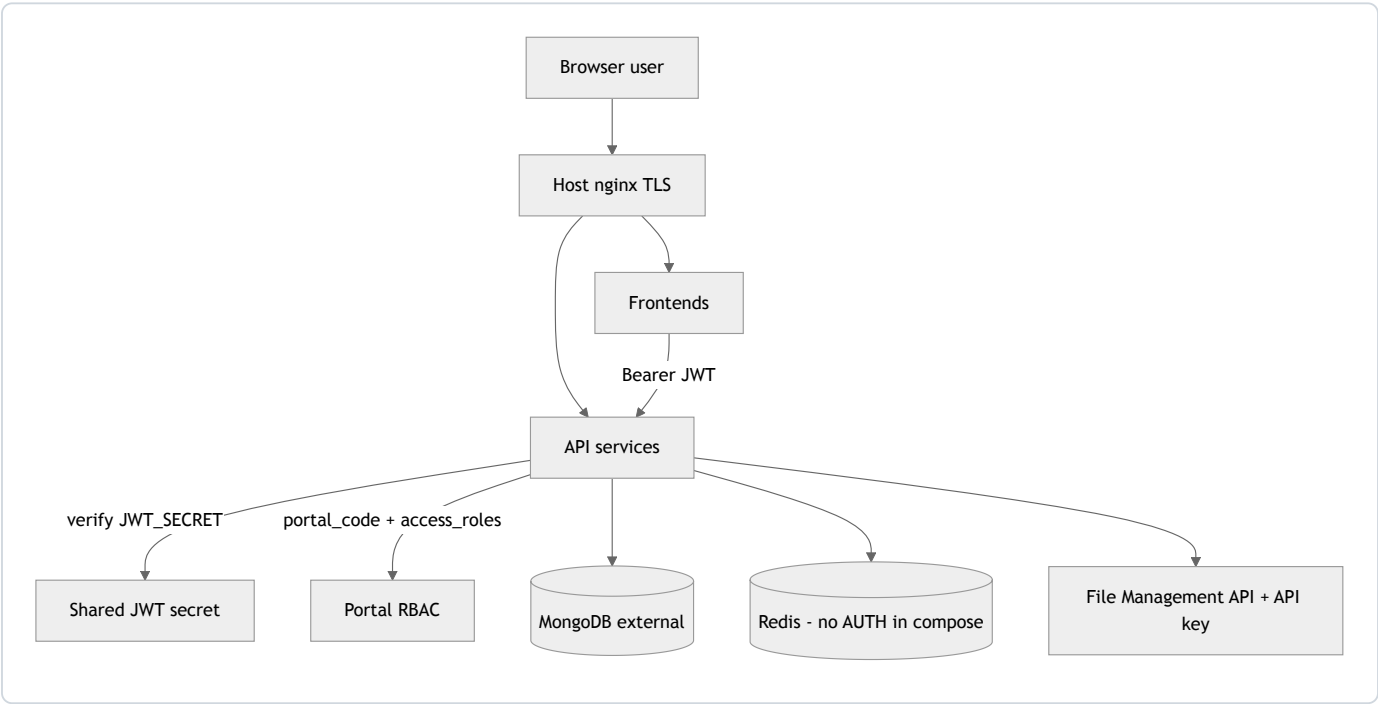
Status: Verified from repository implementation unless marked otherwise.

Scope: Authentication, authorization, secrets, transport security, known risks, and hardening guidance derived from code/config — not a penetration-test report.

Sources:

- `auth-service/` , `opms-backend/src/middlewares/`
- Portal middlewares (lead-manager, work-planner)
- `nginx/` TLS configs
- `docker-compose.yml` , `.env.docker.example` , `DOCKER.md`
- `docs/02-architecture/07-authentication-authorization.md`
- `docs/06-developer-deployment/19-security.md`

1. Security model overview



Trust boundaries

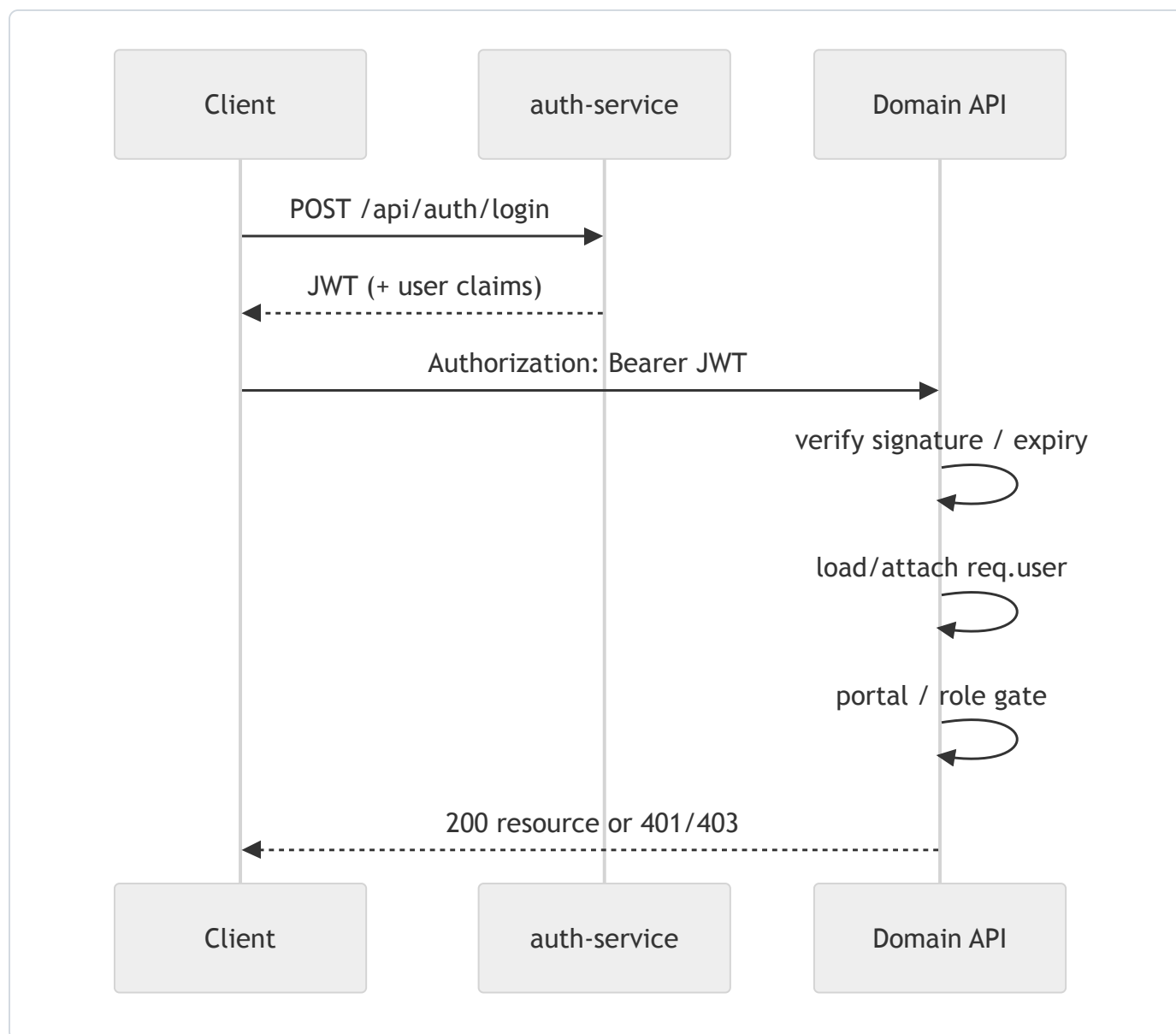
Boundary	What crosses it	Control
Internet → host	HTTPS	nginx TLS + HSTS snippets
Browser → API	REST + JWT	Bearer token; CORS origins
Service → service	Internal HTTP on compose network	Shared JWT; some internal routes unauthenticated
Service → Mongo	Connection string	<code>MONGO_URI</code> secret
Service → Redis	Queue/jobs	No Redis password in current compose
Service → File API	Uploads	<code>FILE_MANAGEMENT_API_KEY</code>

2. Authentication

Mechanism

- **JWT** signed with shared `JWT_SECRET` across services
- Passwords hashed with **bcrypt** (`bcryptjs`)
- Token TTL via `JWT_EXPIRES_IN` (example default `7d`)
- Optional `MASTER_PASSWORD` : break-glass login as any user if set — **must be empty in production**

Login flow



Obtain token

```
POST /api/auth/login
Content-Type: application/json

{ "email": "user@example.com", "password": "*****" }
```

Available on auth-service (`:7003`) or via opms-backend proxy (`:7001/api/auth/login`).

Session introspection

```
GET /api/auth/me
Authorization: Bearer <token>
```

Frontend session handling

App	Primary storage	Notes
app-frontend	<code>shakti.app.session</code> + cookies	SSO hub; launches peers with <code>?token=</code>
opms-frontend	<code>medica.auth</code> + cookies	Consumes SSO token
user / lead / work planner	per-app <code>shakti.*.session</code>	Shared cookie names used for middleware gates

Risk note: JWT in query string for SSO is convenient but can leak via Referer/logs/history. Prefer short-lived handoff and strip token after consume (apps attempt strip after bootstrap).

3. Authorization (RBAC)

Layers

1. **Authenticated** — valid JWT
2. **Portal membership** — `user.portals[]` contains required `portal_code`
3. **access_roles** — role string must match portal gate (admin bypass where coded)
4. **Handler-level checks** — department/workflow rules for specific actions
5. **Soft-delete routes** — authenticated (+ OPMS portal) on many trash/restore endpoints

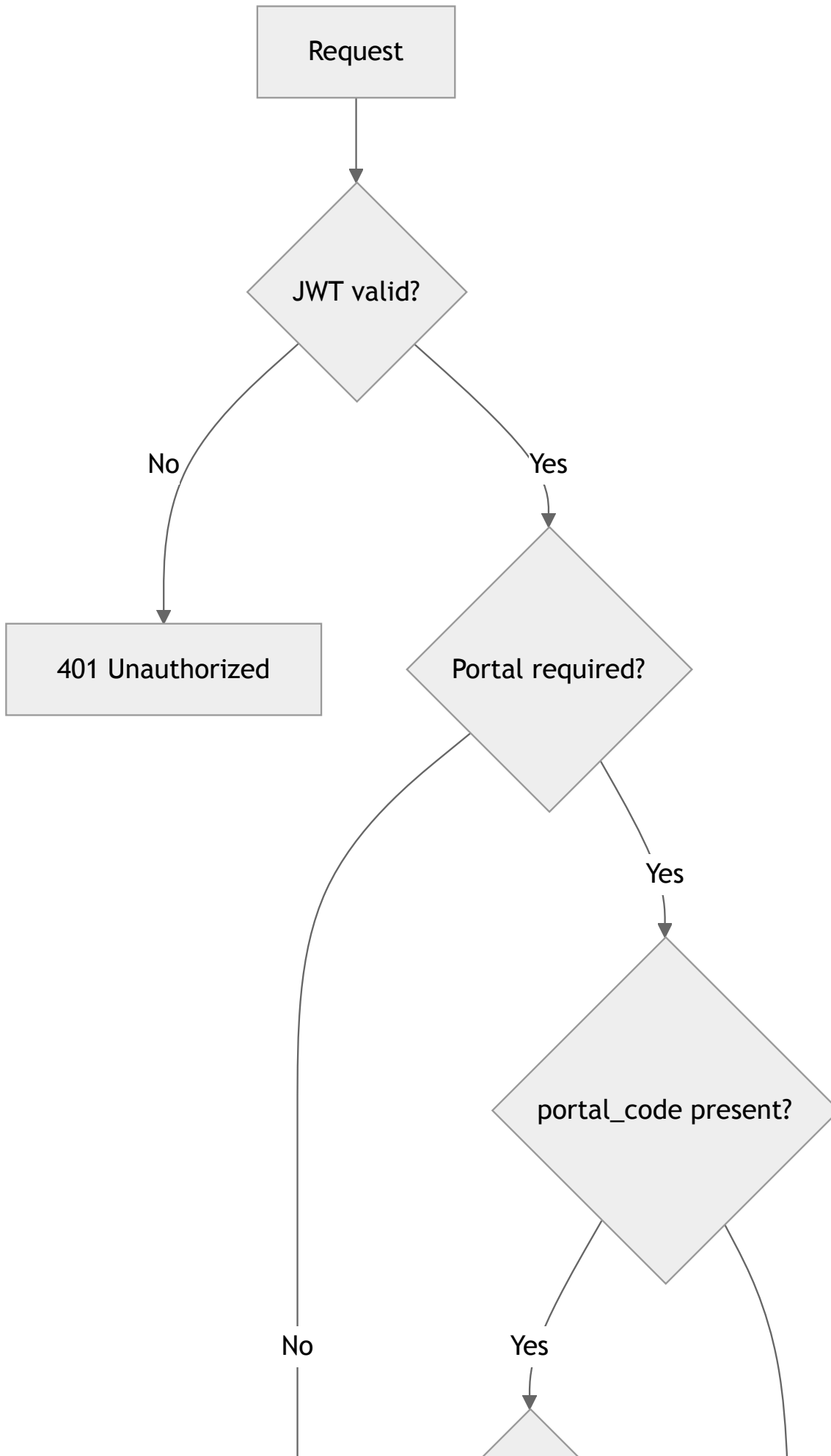
Portal codes and roles (observed)

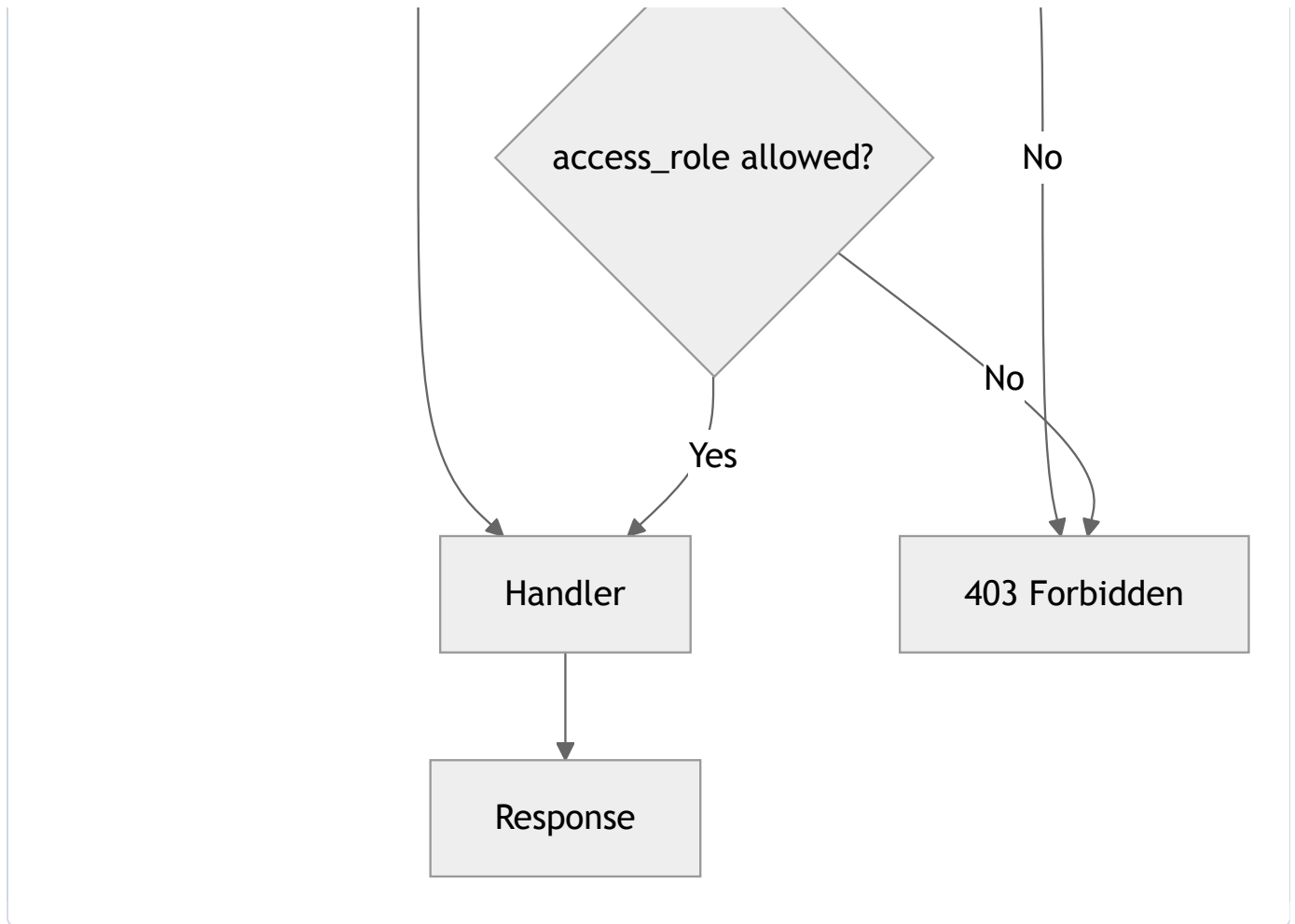
portal_code	Seeded by default?	access_roles (observed)
opms	Yes (DEFAULT_PORTALS)	<code>super_admin</code> , <code>admin</code> , <code>sales</code> , <code>finance</code> , <code>account</code> , <code>dispatch</code>
work_planner	No	<code>admin</code> , <code>manager</code> , <code>executive</code> (+ admin bypasses)
lead_manager	No	<code>admin</code> , <code>manager</code> , <code>executive</code>
user_manager	No	Frontend admin/super-admin style gate — exact role list Unknown

Middleware gates

Gate	Applies to
<code>requireAuth</code>	Most domain routes
<code>requireOpmsAccess</code>	OPMS domain APIs
<code>requireLeadManagerAccess</code>	Lead Manager (most routes)
<code>requireWorkPlannerAccess</code>	Work Planner routes
Webhook secret	Google Sheet ingest

Missing `opms` portal ⇒ **403** even if `department` is set (documented in Swagger + middleware behavior).





4. Transport & edge security

nginx (production pattern)

- TLS 1.2/1.3 via `nginx/snippets/ssl-params.conf`
- HSTS and security headers in ssl snippets
- Let's Encrypt cert paths for `*.spspl.com` / `*.medicaent.in`
- HTTP → HTTPS redirect after cert bootstrap
- SSE-specific proxy settings for notification streams

CORS

Configured via `CORS_ORIGINS` / service CORS settings. Production frontends must be listed. Incorrect CORS is a browser-enforced issue, not a substitute for authz.

Docker network

Services communicate on the Compose bridge using internal URLs (`http://auth-service:7003` , etc.). Assume **network trust inside the compose network** for some internal endpoints (see §6).

5. Secrets & configuration hygiene

Variable	Secret?	Guidance
JWT_SECRET	Yes	Long random; identical across all services
MASTER_PASSWORD	Yes	Leave empty in production
MONGO_URI / MONGODB_URI	Yes	Atlas/user credentials
FILE_MANAGEMENT_API_KEY	Yes	Uploads
SMTP_* / Graph / WhatsApp / VAPID private	Yes	Integrations
GOOGLE_SHEET_WEBHOOK_SECRET	Yes	Webhook auth
SEED_PASSWORD	Yes	Demo only
NEXT_PUBLIC_*	No (public)	Baked into FE builds — never put secrets here

Critical finding

SECRET DETECTED IN SOURCE — NOT REPEATED HERE: `.env.docker.example` appears to contain real-looking credentials (JWT, Mongo URI, API keys, passwords).

Actions required (ops):

1. Rotate all exposed secrets
2. Replace example file with placeholders only
3. Ensure `.env.docker` remains gitignored and never committed

Do not re-declare `MONGO_URI / JWT_SECRET` under Compose `environment:` with empty interpolation — empty values override `env_file` (see `DOCKER.md`).

6. Public and weakly protected surfaces

Documented carefully for threat modeling. **Verified** from route files.

Surface	Auth	Risk
POST <code>/api/auth/login</code>	Public	Brute force — rate limiting not identified
GET <code>/api/company-info</code>	Public (auth-service)	Info disclosure of branding/company fields
Google Sheet webhooks	Shared secret	Secret leakage ⇒ data injection
WhatsApp webhooks	Provider verify	Must validate hub challenge/signature per Meta rules
POST <code>/api/notifications/internal/*</code>	No requireAuth	Anyone who can reach the port can create notifications
opms <code>/api/terms-and-conditions</code> router	No requireAuth on router	Read/write exposure if port reachable
lead-manager <code>/api/attachments</code> router	No requireAuth on router	Attachment abuse if reachable
work-planner attachment view path	Before auth	Public file view redirect — Requires confirmation if intentional



Hardening (recommended, not implemented in-repo):

- Bind internal-only routes to Docker network (do not publish message/notification internal ports publicly where avoidable)
- Add `requireAuth` or shared service token to internal notification/create routes
- Add auth to terms and attachments routers
- Add login rate limiting / lockout
- Enable Redis AUTH and avoid publishing Redis on host

7. Data protection controls present

Control	Status
Password hashing (bcrypt)	Implemented
Soft-delete (recoverable)	Many domain models
Append-only workflow history	OrderWorkflow / status history
HTTPS at edge	nginx configs
File binaries outside app DB	External File Management API
Field-level encryption at rest in app	Not identified
Audit log completeness for all portals	Partial (ActivityLog + workflow)

8. Infrastructure security notes

Item	Finding
Redis	No password in compose; host port <code>7014</code> published — exposure risk if host is reachable
MongoDB	External; protect via Atlas IP allowlists / strong credentials
Frontends	No compose healthchecks; not a direct auth issue
CI/CD secrets scanning	Not identified (no GitHub Actions)
Container user hardening	Default Node images — deep review Not identified

9. Security-related NFRs (from implementation)

Topic	Status
Authentication	JWT
Authorization	Portal RBAC
Confidentiality in transit	TLS at nginx
Rate limiting	Not identified
Centralized security monitoring / WAF	Not identified

Topic	Status
Formal threat model / pen-test artifacts	Not identified

10. Incident checklist (ops-oriented)

1. Rotate `JWT_SECRET` → forces re-login everywhere (coordinate downtime)
2. Rotate Mongo, File API, Graph, WhatsApp, VAPID, webhook secrets
3. Clear `MASTER_PASSWORD` in production env
4. Review nginx access logs for scraping of public/weak routes
5. Restart services after secret rotation: `docker compose --env-file .env.docker up -d`
6. Rebuild frontends if any `NEXT_PUBLIC_*` changed

11. Requires confirmation (product / security team)

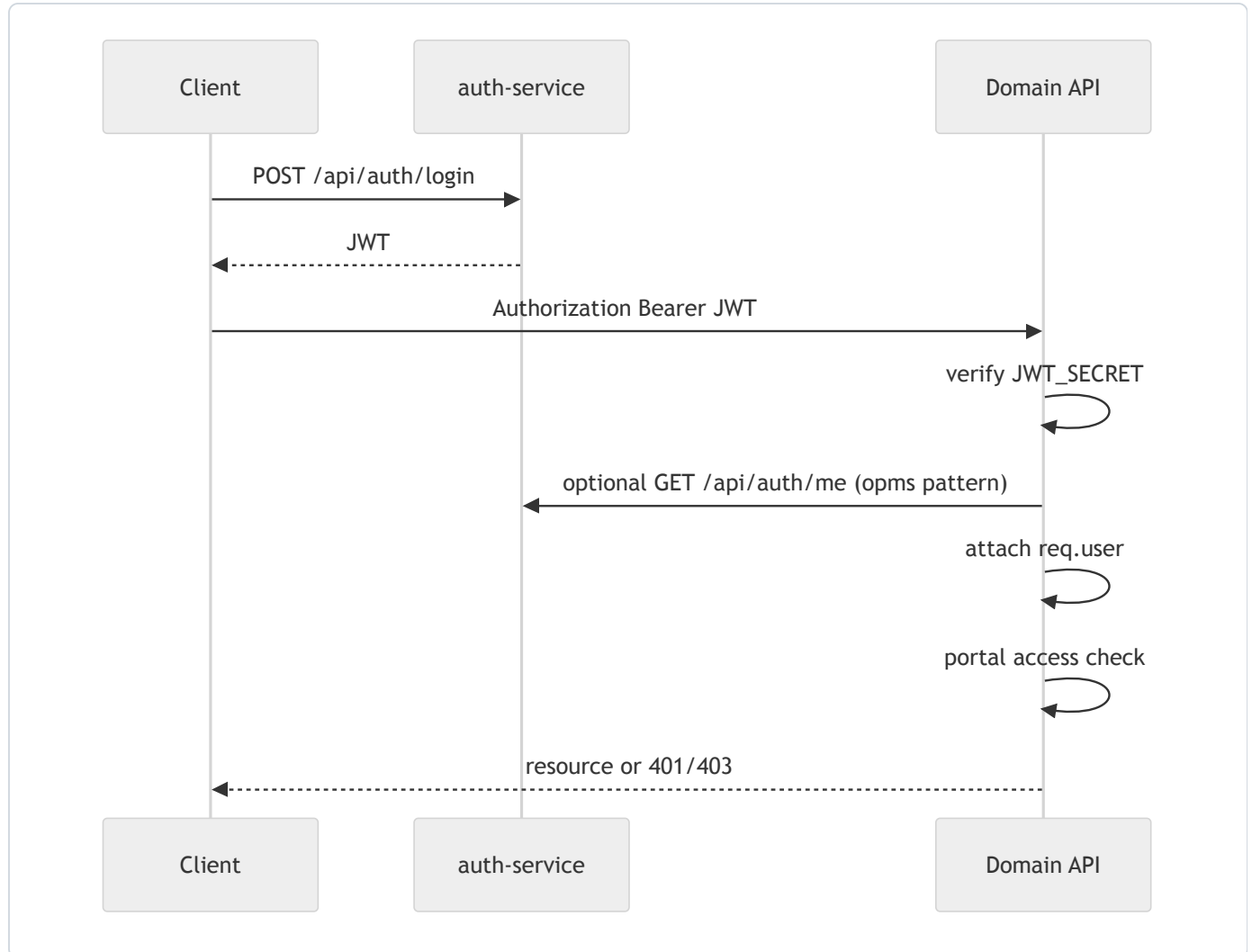
1. Is work-planner public attachment view intentional?
2. Should non- `opms` portals be seeded by default?
3. Is Permission-code RBAC (`opms` Permission model) still a goal vs portal roles only?
4. Target password policy / session length / MFA — **not in codebase**
5. Who owns secret rotation and env hygiene for `.env.docker.example` ?

Related documents

- [Authentication & Authorization \(Architecture\)](#)
- [API Authentication](#)
- [Functional: Authentication](#)
- [Environment Variables](#)
- [Troubleshooting](#)
- [Documentation Audit](#)

Authentication & Authorization

Authentication flow



Token

- Signed with `JWT_SECRET`
- Expiry `JWT_EXPIRES_IN` (example default `7d`)
- Optional `MASTER_PASSWORD` break-glass (**dangerous if set in prod**)

Authorization layers

1. **Authenticated** — valid JWT
2. **Portal membership** — `portals[]` contains required `portal_code`
3. **access_roles** — role string must match portal gate (or admin bypass where coded)
4. **Handler-level checks** — department / role for specific actions (workflow rules, lead admin routes)
5. **Soft-delete permission** — authenticated (+ opms) for trash/restore on some routes; swagger mentions permission codes that may be aspirational vs portal model — **Requires confirmation** for fine-grained permission codes vs portal roles

OPMS roles

super_admin | admin | sales | finance | account | dispatch

Cross-portal

Lead list may allow work_planner access in some routes (`requireLeadManagerOrWorkPlannerAccess`).

Public / weakly protected surfaces (document carefully)

Surface	Notes
POST /api/auth/login	Public
GET /api/company-info	Public on auth-service
Google Sheet webhooks	Secret header/param
WhatsApp webhooks	Public verify/receive
notification internal POSTs	No requireAuth — service trust
opms terms-and-conditions router	No requireAuth on router
lead attachments router	No requireAuth on router
work-planner attachment view	Public view path before auth

See [Security](#).

API Authentication

Method

Bearer JWT in `Authorization: Bearer <token>` header.

Obtain token

```
POST /api/auth/login
Content-Type: application/json

{ "email": "user@example.com", "password": "*****" }
```

Hit auth-service (:7003) or via opms-backend proxy (:7001/api/auth/login).

Session introspection

```
GET /api/auth/me
Authorization: Bearer <token>
```

Authorization

Gate	Applies to
<code>requireAuth</code>	Most domain routes
<code>requireOpmsAccess</code>	OPMS domain on opms-backend
<code>requireLeadManagerAccess</code>	Lead Manager mutating/list routes
<code>requireWorkPlannerAccess</code>	Work Planner routes
Webhook secrets	Google Sheet / WhatsApp verify

Portal requirement (OPMS)

User must include `portals[]` entry `{ portal_code: "opms", access_roles: [...] }`.

Related

- [Architecture Auth](#)

Module: Authentication

Purpose

Authenticate users, issue JWTs, expose current user profile, and support password change.

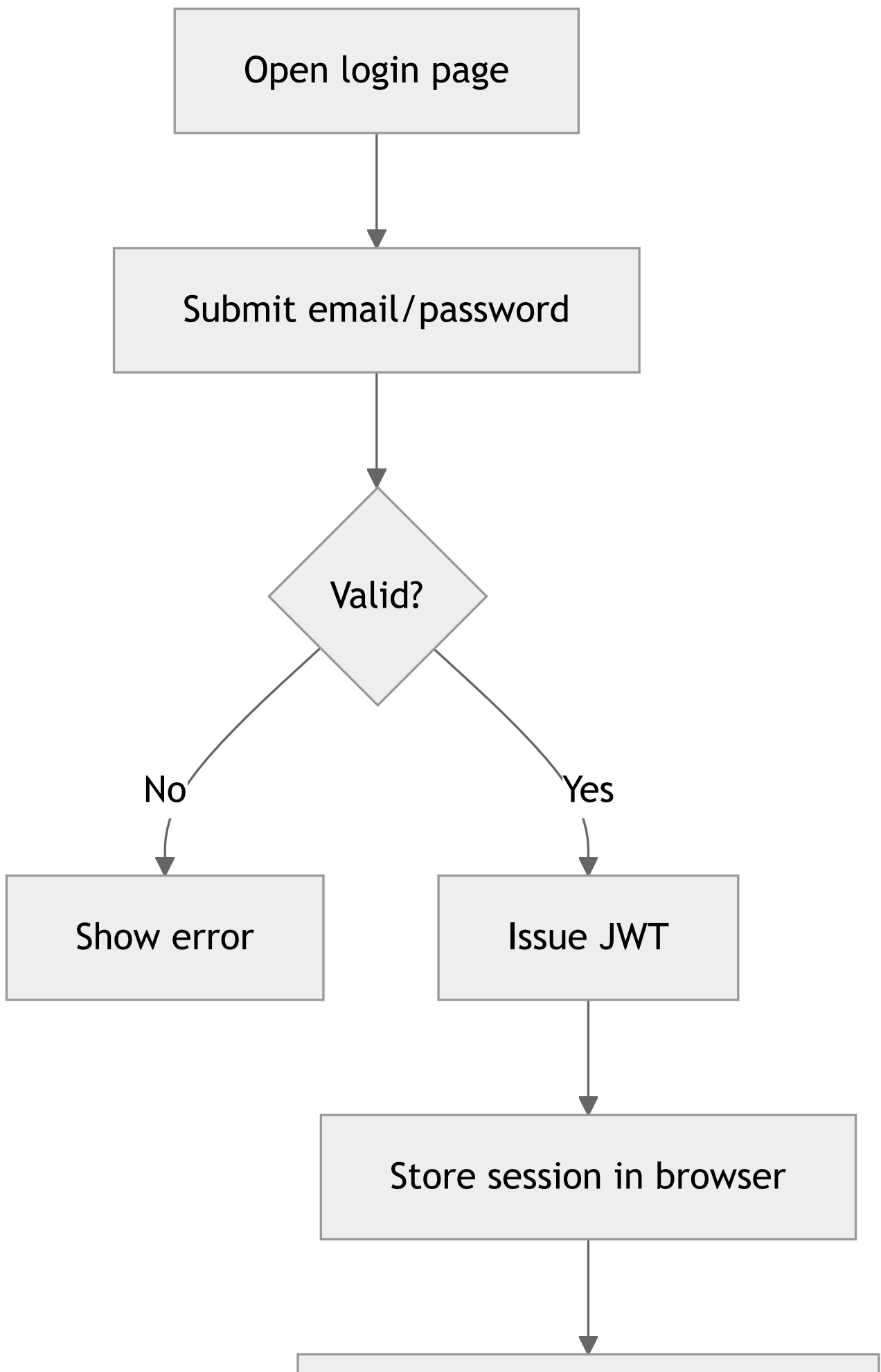
Actors

Any user with an active auth-service account; App Frontend and all portal frontends.

Preconditions

- auth-service running and Mongo reachable
- User exists with password hash; `is_active` true (**Inferred** inactive users rejected — verify service)

Workflow



Navigate dashboard/portal



Optional SSO ?token= to
other app

Business Rules

- Shared `JWT_SECRET` across services
- Optional `MASTER_PASSWORD` can authenticate as any user if set — **must be empty in production**

Validations

Email/password required on login (controller-level).

Permissions

Login public; `/me` and change-password require auth.

Exceptions / Errors

401 invalid credentials; 401/403 on protected routes without/with wrong token.

Related APIs

- `POST /api/auth/login`
- `GET /api/auth/me`
- `POST /api/auth/change-password`

Related DB

User, Role, Portal, Department, CompanyInfo (auth-service)

Related UI

app-frontend `/`, portal `/login` pages

Dependencies

auth-service; MongoDB; frontends

Source

```
auth-service/src/modules/auth/
```

Security

Implemented

- JWT auth + bcrypt passwords
- Portal RBAC
- HTTPS via nginx TLS configs
- Webhook secrets for Google Sheets
- Soft-delete vs hard-delete for many records

Risks identified in codebase (document, do not "fix" here)

Risk	Notes
Secrets in <code>.env.docker.example</code>	Appears to contain real credentials — rotate & sanitize
<code>MASTER_PASSWORD</code>	Break-glass; must be empty in prod
Redis without AUTH + published port	Network exposure risk
Notification internal routes unauthenticated	Network trust assumed
opms terms-and-conditions router without <code>requireAuth</code>	Exposure risk
lead-manager attachments router without <code>requireAuth</code>	Exposure risk
work-planner attachment view before auth	Intentional public view? Requires confirmation
Duplicated schema registries	Drift / authz inconsistency risk

Rate limiting

Not identified.

Related

- [Auth architecture](#)

Environment Variables

IMPORTANT: Never copy real secrets into documentation.

SECRET DETECTED IN SOURCE — NOT INCLUDED IN DOCUMENTATION: `.env.docker.example` currently contains values that look like live JWT secrets, Mongo credentials, API keys, and passwords. Treat as a credential-hygiene incident; rotate if exposed; replace example file with placeholders.

Variable	Required	Purpose	Example (placeholder)	Secret
NODE_ENV	yes	Runtime mode	production	no
JWT_SECRET	yes	JWT signing	change-me-long-random	yes
JWT_EXPIRES_IN	no	Token TTL	7d	no
MASTER_PASSWORD	no	Break-glass login	empty in prod	yes
JSON_BODY_LIMIT	no	Body size	50mb	no
MONGO_URI / MONGODB_URI	yes	Mongo connection	mongodb+srv://USER:PASS@host/db	yes
MONGODB_LOOKUP_FAMILY	no	Prefer IPv4	4	no
REDIS_URL	no	Redis	redis://redis:6379	no*
REDIS_PORT	no	Host publish	7014	no
*_PORT	no	Host ports	see compose defaults	no
AUTH_SERVICE_URL etc.	no	Internal URLs	http://auth-service:7003	no
API_PUBLIC_BASE_URL	prod	Public API	https://api-opms.example.com	no
NEXT_PUBLIC_*	prod	Browser URLs (build-time)	https://...	no
CORS_ORIGINS	prod	Extra CORS	comma URLs	no
COMPANY_NAME / LOGO	no	Branding fallback	—	no
SEED_*	no	Seed controls	—	SEED_PASSWORD yes
FOLLOWUP_REMINDER_*	no	Lead cron	—	no
FILE_MANAGEMENT_API_URL/KEY	optional	Uploads	—	key yes
SMTP_*	optional	Email	—	pass yes
MICROSOFT_GRAPH_*	optional	Email	—	secret yes
WHATSAPP_*	optional	WhatsApp	—	yes

Variable	Required	Purpose	Example (placeholder)	Secret
VAPID_*	optional	Web push	—	private yes
GOOGLE_SHEET_WEBHOOK_SECRET	optional	Webhooks	—	yes
ADMIN_EMAIL / ACCOUNT_EMAIL / FINANCE_EMAIL / DUE_SHEET_EMAIL / DISPATCH_EMAIL	optional	Role mailboxes	—	no
COMPOSE_PROJECT_NAME / ENV_FILE	no	Multi-stack	—	no
APP_LOGIN_URL / LEAD_MANAGER_PUBLIC_URL / FRONTEND_URL	optional	Links in emails	—	no

* Redis currently unauthenticated in compose.

Source: `.env.docker.example` , service `env.js` files, compose.